

Manifest File

The android manifest file is the main file which holds the core information of the application. Like the package name and details, Application, Activities, Services, Broadcast, Receiver, Content Provider ...

Main task of this file

- It is **responsible to protect the application** to access any protected parts by providing the permissions.
- It also **declares the android api** that the application is going to use.
- It **lists the instrumentation classes**. The instrumentation classes provides profiling and other informations. These informations are removed just before the application is published etc.

This is the required xml file for all the android application and located inside the root directory.

A Sample AndroidManifest.xml file

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.maskeyconsultancy.db" >
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name"
            android:theme="@style/AppTheme.NoActionBar" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".UserForm"
            android:label="User Form" />
        <activity
            android:name=".DataListActivity"
            android:label="@string/title_activity_data_list"
            android:theme="@style/AppTheme.NoActionBar" >
        </activity>
    </application>
</manifest>
```

Element of AndroidManifest.xml File

<manifest>

manifest is the root element of the AndroidManifest.xml file. It has **package** attribute that describes the package name of the activity class

<application>

application is the subelement of the manifest. It includes the namespace declaration. This element contains several subelements that declares the application component such as activity etc.

The commonly used attributes are of this element are **icon, label, theme** etc.

android:icon represents the icon for all the android application components.

android:label works as the default label for all the application components.

android:theme represents a common theme for all the android activities.

<activity>

activity is the subelement of application and represents an activity that must be defined in the AndroidManifest.xml file. It has many attributes such as label, name, theme, launchMode etc.

android:label represents a label i.e. displayed on the screen.

android:name represents a name for the activity class. It is required attribute.

<intent-filter>

intent-filter is the sub-element of activity that describes the type of intent to which activity, service or broadcast receiver can respond to.

<action>

It adds an action for the intent-filter. The intent-filter must have at least one action element

<category>

It adds a category name to an intent-filter.

Activity Life Cycle

An activity is an application component that provides a screen with which android user can interact. Activity will hold many sub components to work with programs such as dial, taking photo, view map etc...

An application consist of multiple activities those can be linked with actions. Typically default first activity is called Main activity which is presented to the user when application is launched. If same application has more activities then other activities remain stopped except the one that is launched.

Previous activity from where the current activity has opened will remain in “Back stack” that will be executed when user hit's back button.

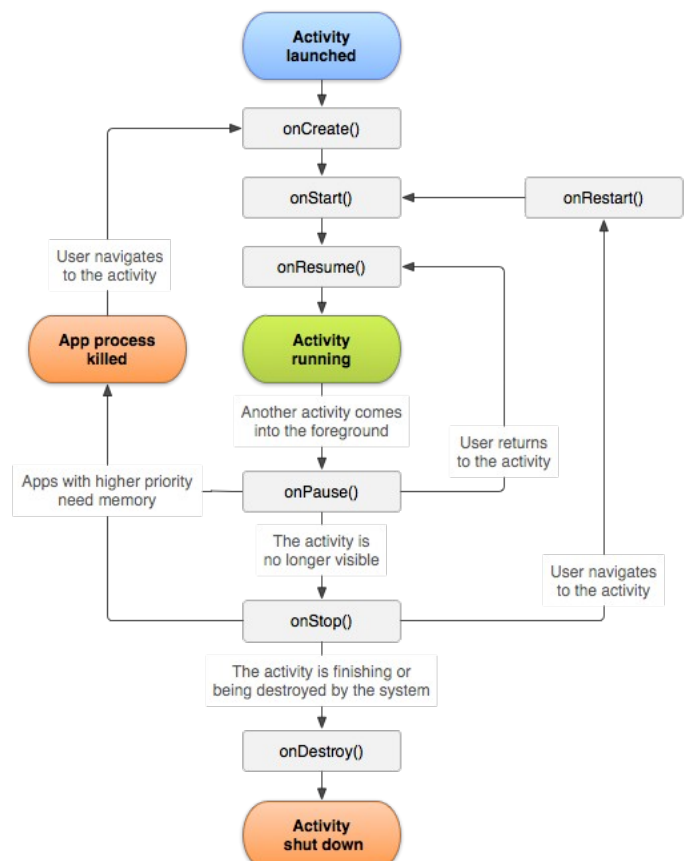
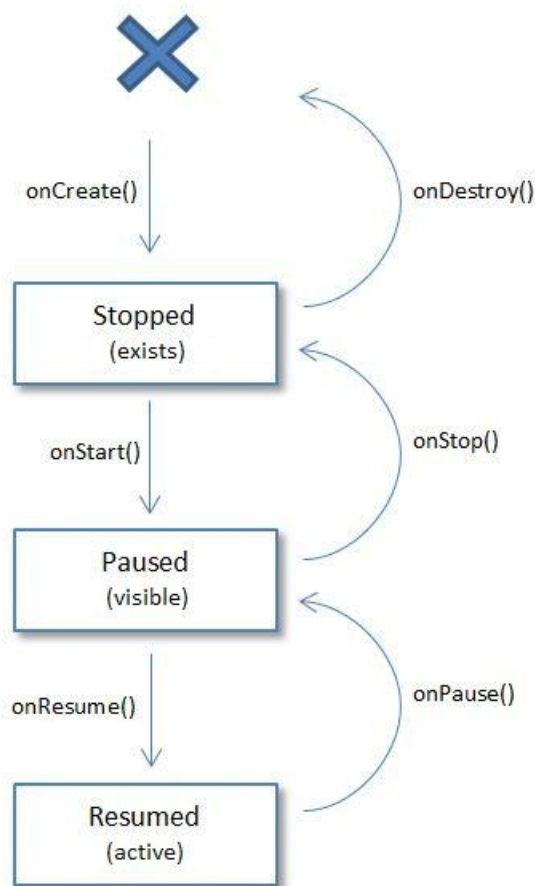
When activities are changed then activity lifecycle callback method will notify the change to the state.

When application is launched then activity could have 3 states in it's lifecycle

Resumed- Activity is visible on the screen, it is focused and the user can interact with it. This state is sometimes called Running

Paused- Activity is not focused, user cannot interact with it, but it is visible (it is covered by another Activity, which doesn't cover the whole screen or is transparent)

Stopped- Activity is not visible (is completely covered by another Activity), that is, it is not focused and user cannot interact with it



Extending Activity Class

For each new activity, we can extend the main Activity. Once we do this we can use all the method and properties of main activity. Suppose if you have 7,8,9 in main activity you can extend that as below

```
public class MyActivity extends Activity {...}
```

In this example “MyActivity” is new activity which has extended the base Activity.

The Intent Class

An intent is an abstract description of an operation to be performed. It can be used with [startActivity](#) to launch an [Activity](#), [broadcastIntent](#) to send it to any interested [BroadcastReceiver](#) components, and [startService\(Intent\)](#) or [bindService\(Intent, ServiceConnection, int\)](#) to communicate with a background [Service](#).

An Intent provides a facility for performing late runtime binding between the code in different applications. Its most significant use is in the launching of activities, where it can be thought of as the glue between activities. It is basically a passive data structure holding an abstract description of an action to be performed.

An [Intent](#) is a messaging object you can use to request an action from another [app component](#). Although intents facilitate communication between components in several ways, there are three fundamental use-cases:

To start an activity

An [Activity](#) represents a single screen in an app. You can start a new instance of an [Activity](#) by passing an [Intent](#) to [startActivity\(\)](#). The [Intent](#) describes the activity to start and carries any necessary data.

If you want to receive a result from the activity when it finishes, call [startActivityForResult\(\)](#). Your activity receives the result as a separate [Intent](#) object in your activity's [onActivityResult\(\)](#) callback. For more information, see the [Activities](#) guide.

To start a service

A [Service](#) is a component that performs operations in the background without a user interface. You can start a service to perform a one-time operation (such as download a file) by passing an [Intent](#) to [startService\(\)](#). The [Intent](#) describes the service to start and carries any necessary data.

If the service is designed with a client-server interface, you can bind to the service from another component by passing an [Intent](#) to [bindService\(\)](#).

To deliver a broadcast

A broadcast is a message that any app can receive. The system delivers various broadcasts for system events, such as when the system boots up or the device starts charging. You can deliver a broadcast to other apps by passing an [Intent](#) to [sendBroadcast\(\)](#), [sendOrderedBroadcast\(\)](#), or [sendStickyBroadcast\(\)](#).

Intent Types

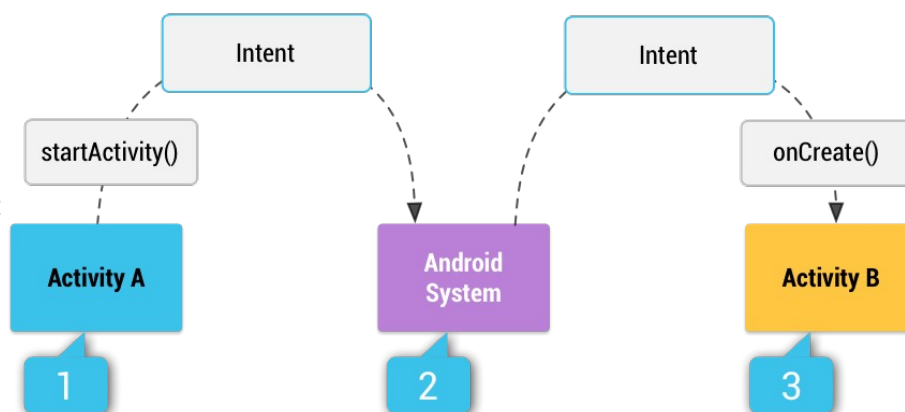
There are two types of intents:

Explicit intents specify the component to start by name (the fully-qualified class name). You'll typically use an explicit intent to start a component in your own app, because you know the class name of the activity or service you want to start. For example, start a new activity in response to a user action or start a service to download a file in the background.

Implicit intents do not name a specific component, but instead declare a general action to perform, which allows a component from another app to handle it. For example, if you want to show the user a location on a map, you can use an implicit intent to request that another capable app show a specified location on a map.

When you create an explicit intent to start an activity or service, the system immediately starts the app component specified in the [Intent](#) object.

Figure 1.
Illustration of how an implicit intent is delivered through the system to start another activity:



[1] *Activity A* creates an [Intent](#) with an action description and passes it to `startActivity()`.

[2] The Android System searches all apps for an intent filter that matches the intent. When a match is found,

[3] the system starts the matching activity (*Activity B*) by invoking its `onCreate()` method and passing it the [Intent](#).

When you create an implicit intent, the Android system finds the appropriate component to start by comparing the contents of the intent to the *intent filters* declared in the [manifest file](#) of other apps on the device. If the intent matches an intent filter, the system starts that component and delivers it the [Intent](#) object. If multiple intent filters are compatible, the system displays a dialog so the user can pick which app to use.

An intent filter is an expression in an app's manifest file that specifies the type of intents that the component would like to receive. For instance, by declaring an intent filter for an activity, you make it possible for other apps to directly start your activity with a certain kind of intent. Likewise, if you do *not* declare any intent filters for an activity, then it can be started only with an explicit intent.

Caution: To ensure your app is secure, always use an explicit intent when starting a [Service](#) and do not declare intent filters for your services. Using an implicit intent to start a service is a security hazard because you cannot be certain what service will respond to the intent, and the user cannot see which service starts. Beginning with Android 5.0 (API level 21), the system throws an exception if you call [bindService\(\)](#) with an implicit intent.

Building an Intent

An [Intent](#) object carries information that the Android system uses to determine which component to start (such as the exact component name or component category that should receive the intent), plus information that the recipient component uses in order to properly perform the action (such as the action to take and the data to act upon).

The primary information contained in an [Intent](#) is the following:

Component name

The name of the component to start.

This is optional, but it's the critical piece of information that makes an intent **explicit**, meaning that the intent should be delivered only to the app component defined by the component name. Without a component name, the intent is **implicit** and the system decides which component should receive the intent based on the other intent information (such as the action, data, and category—described below). So if you need to start a specific component in your app, you should specify the component name.

Note: When starting a [Service](#), you should **always specify the component name**. Otherwise, you cannot be certain what service will respond to the intent, and the user cannot see which service starts.

This field of the [Intent](#) is a [ComponentName](#) object, which you can specify using a fully qualified class name of the target component, including the package name of the app. For example, `com.example.ExampleActivity`. You can set the component name with [setComponent\(\)](#), [setClass\(\)](#), [setClassName\(\)](#), or with the [Intent](#) constructor.

Action

A string that specifies the generic action to perform (such as *view* or *pick*).

In the case of a broadcast intent, this is the action that took place and is being reported. The action largely determines how the rest of the intent is structured—particularly what is contained in the data and extras.

You can specify your own actions for use by intents within your app (or for use by other apps to invoke components in your app), but you should usually use action constants defined by the [Intent](#) class or other framework classes. Here are some common actions for starting an activity:

ACTION_VIEW

Use this action in an intent with `startActivity()` when you have some information that an activity can show to the user, such as a photo to view in a gallery app, or an address to view in a map app.

ACTION_SEND

Also known as the "share" intent, you should use this in an intent with `startActivity()` when you have some data that the user can share through another app, such as an email app or social sharing app.

See the [Intent](#) class reference for more constants that define generic actions. Other actions are defined elsewhere in the Android framework, such as in [Settings](#) for actions that open specific screens in the system's Settings app.

You can specify the action for an intent with `setAction()` or with an [Intent](#) constructor.

If you define your own actions, be sure to include your app's package name as a prefix. For example:

```
static final String ACTION_TIMETRAVEL = "com.example.action.TIMETRAVEL";
```

Data

The URI (a [Uri](#) object) that references the data to be acted on and/or the MIME type of that data. The type of data supplied is generally dictated by the intent's action. For example, if the action is [ACTION_EDIT](#), the data should contain the URI of the document to edit.

When creating an intent, it's often important to specify the type of data (its MIME type) in addition to its URI. For example, an activity that's able to display images probably won't be able to play an audio file, even though the URI formats could be similar. So specifying the MIME type of your data helps the Android system find the best component to receive your intent. However, the MIME type can sometimes be inferred from the URI—particularly when the data is a [content](#): URI, which indicates the data is located on the device and controlled by a [ContentProvider](#), which makes the data MIME type visible to the system.

To set only the data URI, call `setData()`. To set only the MIME type, call `setType()`. If necessary, you can set both explicitly with `setDataAndType()`.

Caution: If you want to set both the URI and MIME type, **do not** call `setData()` and `setType()` because they each nullify the value of the other. Always use `setDataAndType()` to set both URI and MIME type.

Fragments

A [Fragment](#) represents a behavior or a portion of user interface in an [Activity](#). You can combine multiple fragments in a single activity to build a multi-pane UI and reuse a fragment in multiple activities. You can think of a fragment as a modular section of an activity, which has its own lifecycle, receives its own input events, and which you can add or remove while the activity is running (sort of like a "sub activity" that you can reuse in different activities).

A fragment must always be embedded in an activity and the fragment's lifecycle is directly affected by the host activity's lifecycle. For example, when the activity is paused, so are all fragments in it, and when the activity is destroyed, so are all fragments. However, while an activity is running (it is in the *resumed* [lifecycle state](#)), you can manipulate each fragment independently, such as add or remove them. When you perform such a fragment transaction, you can also add it to a back stack that's managed by the activity—each back stack entry in the activity is a record of the fragment transaction that occurred. The back stack allows the user to reverse a fragment transaction (navigate backwards), by pressing the *Back* button.

When you add a fragment as a part of your activity layout, it lives in a [ViewGroup](#) inside the activity's view hierarchy and the fragment defines its own view layout. You can insert a fragment into your activity layout by declaring the fragment in the activity's layout file, as a `<fragment>` element, or from your application code by adding it to an existing [ViewGroup](#). However, a fragment is not required to be a part of the activity layout; you may also use a fragment without its own UI as an invisible worker for the activity.

This document describes how to build your application to use fragments, including how fragments can maintain their state when added to the activity's back stack, share events with the activity and other fragments in the activity, contribute to the activity's action bar, and more.